MediCare
Technical Project Report

Implementation Documentation, Test Results, Challenges, and Outcomes

Project Type: Capstone / Full-stack AI medical web application
Prepared from repository artifacts in Cancer_detection-main.zip

Course: Capstone II

Name: Hitanshu Bhatt

Student ID: 00367185
Date: April 15, 2026

Frontend	HTML, CSS, JavaScript doctor and patient workflows
Backend	FastAPI service with MySQL-backed booking and patient records
Machine Learning	PyTorch EfficientNet-B0 4-class lung image classifier
Primary Outcome	Integrated prototype for scan prediction plus clinic workflow

Table of Contents

1. Project Summary

Medicare is a full-stack capstone application that combines a deep learning image classifier with a hospital-style workflow for patient registration, appointment booking, and doctor-side review. The repository includes a FastAPI backend, a MySQL connection layer, a saved PyTorch model, model training scripts, and a multi-page vanilla JavaScript frontend.

From a technical standpoint, the project demonstrates successful cross-layer integration: a browser-based scan upload page sends multipart image data to a prediction API, the backend loads a trained EfficientNet-B0 model once at startup, and the result is returned to the user as a predicted cancer class with a confidence score. In parallel, appointment and patient management are handled through database-driven forms and dashboard views.

The main outcome is a working prototype architecture rather than a production-ready clinical system. The repository shows strong evidence of end-to-end intent and meaningful implementation progress. It also shows several unfinished integration points, especially around route consistency, formal testing, and operational hardening. Those gaps are documented explicitly in this report so the technical evaluation remains accurate and credible.

2. Implementation Explanation

The system is organized into three cooperating layers: frontend presentation logic, FastAPI backend services, and a machine learning inference component backed by a trained EfficientNet model file. The data flow is request-driven: forms or image uploads originate in the browser, are transmitted over HTTP to FastAPI endpoints, and either read/write relational data in MySQL or invoke the inference pipeline.

2.1 Frontend construction

The frontend is built with plain HTML, CSS, and JavaScript. Separate pages exist for landing, doctor login, doctor dashboard, patient registration, appointment management, public booking, and cancer detection. JavaScript files handle DOM events, browser-side validation, API calls through fetch(), and dynamic rendering of appointment cards or patient tables.

This approach keeps the frontend simple and framework-free. The application state is mostly transient and page-local, with a small amount of session-like behavior implemented through localStorage. For example, after login the doctor's name is stored locally and later displayed in the dashboard header.

2.2 Backend service construction

The backend is written in Python with FastAPI. CORS is configured to allow any frontend origin, which simplifies development when HTML pages are opened directly in the browser or served from a different local port. The backend exposes routes for doctor signup, public appointment booking, patient creation, appointment retrieval, and lung cancer prediction.

Two database access patterns appear in the repository. Some routes use mysql.connector synchronously through get_db_connection(), while the signup flow uses aiomysql asynchronously. This mixed access pattern works conceptually, but it also introduces maintenance overhead because error handling, transaction management, and connection lifecycle behavior are not standardized across the whole backend.

2.3 Machine learning pipeline

The machine learning component uses PyTorch and torchvision's EfficientNet-B0 architecture. Training scripts resize images to 224×224 pixels, convert them to tensors, normalize them with ImageNet channel statistics, and train a four-class classifier by replacing the final EfficientNet layer with a linear layer sized for four outputs.

At inference time, the backend prediction service loads the saved model weights, applies the same resize and normalization steps, performs a forward pass inside torch.no_grad(), and converts raw logits into class probabilities using softmax. The highest-probability index is mapped back to one of four clinical labels: adenocarcinoma, large cell carcinoma, normal, or squamous cell carcinoma.

2.4 Data flow

The most important end-to-end path is the scan prediction flow. A user selects an image file in detection.html. detection.js previews the image, packages it in FormData, and posts it to /predict-lung-cancer. The FastAPI

router hands the uploaded file to the prediction service, which opens the image with PIL, preprocesses it, runs the model, and returns a JSON response containing prediction and confidence.

The second major flow is appointment handling. In the public booking flow, the user submits an email, desired appointment timestamp, and optional reason. The backend checks whether a matching patient record exists, selects a doctor, validates that the time slot is free, inserts a new appointment row, and commits the transaction. In the doctor workflow, appointment data is retrieved through a JOIN between appointments and patients so patient names can be displayed directly on the page.

2.5 Integration approach

Integration is REST-style and page oriented. Each frontend page directly targets one or more backend endpoints. The design is straightforward: no message broker, no background job queue, and no frontend state framework are used. This is appropriate for a capstone prototype because it makes the call chain easy to explain and debug.

The repository also reveals an important integration lesson: the frontend and backend evolved at different speeds. Several frontend files reference routes such as /login, /patients, /register-patient, /add-appointment, and /get-today-appointments, but those routes are not currently present in the backend source included in the zip. As a result, the architectural design is coherent, but the integrated build is only partially synchronized at code level.

2.6 Key logic excerpts

Backend route structure

```
@app.post("/signup")
async def register_doctor(user: UserSignup):
    hashed_pw = hashlib.sha256(user.password.encode()).hexdigest()
    ... INSERT INTO users (... role = "Doctor" ...)

@router.post("/predict-lung-cancer")
async def predict_route(file: UploadFile = File(...)):
    result = await predict_lung_cancer(file)
```

Prediction algorithm core

```
image = Image.open(file.file).convert("RGB")
image = transform(image).unsqueeze(0)
with torch.no_grad():
    outputs = model(image)
    probabilities = torch.softmax(outputs, dim=1)
    predicted = torch.argmax(probabilities, dim=1)
```

3. Technical Inventory and Outcomes

The repository contains enough artifacts to show the intended scope of the project. The training dataset is organized into train, validation, and test folders, each with four labeled classes. The application also contains real interface screenshots, a saved .pth model file, and separated frontend/backend directories, which together provide evidence of a complete prototype effort.

Artifact	Observed value

Train images	613
Validation images	72
Test images	315
Classes	4
Python backend files	7 major .py files
Frontend pages/scripts/styles	multiple HTML/JS/CSS assets

Outcome-wise, the project achieves three meaningful technical goals. First, it proves that a trained computer vision model can be exposed through a user-facing web interface. Second, it connects that AI feature to a broader healthcare workflow instead of presenting the model in isolation. Third, it demonstrates practical software engineering decisions such as modular inference code, browser-based API communication, and relational persistence for patient-facing records.

4. Test Results Documentation

Testing evidence in the repository is limited, so this section distinguishes between observed evidence and missing evidence. The goal is accuracy: only results that can be reconstructed from the project files are treated as verified. No formal pytest suite, no CI pipeline, and no benchmark report were included in the uploaded zip.

The repository includes one lightweight script, training/test.py, which reports PyTorch version information and whether CUDA is available. That script validates runtime environment visibility only; it does not validate functional correctness of routes, database operations, or model accuracy.

Test area	Evidence found	Result	Assessment
Environment check	training/test.py	PyTorch/CUDA presence can be printed	Very limited coverage
Frontend–backend route alignment	JS fetch targets vs backend routes	3 of 8 referenced frontend API paths are present	Partial integration
Dataset readiness	Train/validation/test folders present	613/72/315 images observed across splits	Sufficient for prototype experimentation
Model deployment readiness	Saved lung_cancer_model.pth included	Inference artifact present	Positive deployment indicator
Automated unit tests	No pytest/unittest suite found	0 formal automated tests	Coverage gap

Reconstructed test coverage metric: based on route references in the frontend code, 8 unique backend endpoints are expected by the user interface, while only 3 matching paths are implemented in the backend source in this archive (/signup, /get-appointments, and /predict-lung-cancer). This yields an observed route alignment of 37.5%. This is not a traditional code-coverage number, but it is a useful integration coverage metric for the current repository snapshot.

Observed performance benchmark evidence is also limited. No load-testing scripts, timing harnesses, or model latency logs are included. The presence of EfficientNet-B0 and 224×224 preprocessing suggests the model is lightweight enough for single-image inference in a prototype setting, but the repository does not provide measured latency, throughput, or concurrent request data. Therefore any production-performance claim would be unsupported.

Validation evidence that can be reasonably accepted includes application screenshots, the trained model file, structured dataset folders, and code paths that implement upload, preprocess, infer, and respond behavior. Together, these indicate that the system was run and visualized at least in a development environment.

Defect log and resolutions

Defect / risk	Evidence	Attempted / implied approach	Current resolution status
Missing /login endpoint	login.js calls /login but backend route absent	Frontend prepared for credential submission	Not resolved in current backend snapshot
Patient route mismatch	patients.js uses /register-patient and /patients; backend has /create-patient only	Partial renaming occurred during development	Needs API contract unification
Appointment creation route missing	appointments.js posts to /add-appointment	Doctor scheduling UI expects CRUD behavior	Needs implementation or frontend update
Dashboard today endpoint missing	detection.js references /get-today-appointments	Separate dashboard widget planned	Needs backend query endpoint
Hard-coded DB credentials	backend/main.py and database.py contain root credentials	Simplified local development configuration	Should move to environment variables

5. Challenges and Solutions

Several technical challenges are visible from the codebase, and they are typical of projects that combine AI, backend APIs, databases, and frontend pages in a short capstone timeline.

Model integration into a web app

Problem: The project had to move from a training script into a usable browser workflow.

What worked: The solution that worked was to isolate inference inside a prediction service, preload the model, and expose a dedicated FastAPI route that accepts uploaded files.

Synchronizing frontend and backend development

Problem: Different pages were built quickly and started depending on route names before the backend API contract had fully stabilized.

What worked: What ultimately worked partially was page-specific fetch logic plus a minimal backend; however, unresolved endpoint mismatches remain, showing that interface contracts need to be frozen earlier in future iterations.

Handling relational booking logic

Problem: Booking is not just a form insert; it requires patient lookup, doctor lookup, schedule conflict detection, and transaction commit/rollback behavior.

What worked: The implemented public booking route solves this by checking for an existing patient, retrieving a doctor, verifying slot availability, then inserting only when the slot is free.

Balancing prototype speed and security

Problem: The application hashes doctor passwords, but it still uses open CORS and hard-coded database credentials for convenience.

What worked: The practical short-term solution was to keep setup simple during development. The lesson for future projects is to introduce .env-based secrets and tighter origin controls before deployment.

A further lesson concerns testing maturity. The repository clearly prioritizes feature delivery and demonstration value over automated verification. That is common in student projects, but it creates risk when the application grows. Future work should add API tests for each route, sample inference tests with known images, and database integration tests using a seeded local schema.

6. Overall Outcomes

Overall, the project succeeds as a technically ambitious prototype. It combines machine learning, backend development, relational data handling, and multi-page frontend implementation in a single healthcare-themed system. The clearest success is architectural breadth: the project is much more than a standalone model notebook.

The strongest completed outcome is the AI inference path, which is modular, understandable, and aligned with the training preprocessing logic. The strongest partially completed outcome is the clinical workflow layer, where the interface design and database-driven intent are clear but some endpoint contracts are not yet aligned in code. The weakest area is formal testing, especially benchmark evidence and repeatable functional validation.

For future improvement, the top priorities should be: unify endpoint names across frontend and backend; add missing doctor workflow routes; move secrets to environment variables; add automated route tests and seed-data integration tests; and document measured model accuracy and latency from a reproducible evaluation script.

7. Reference Images

The repository contains screenshots showing the implemented user interface. A combined figure is included below as visual evidence of the dashboard, detection page, appointment view, and public booking page.

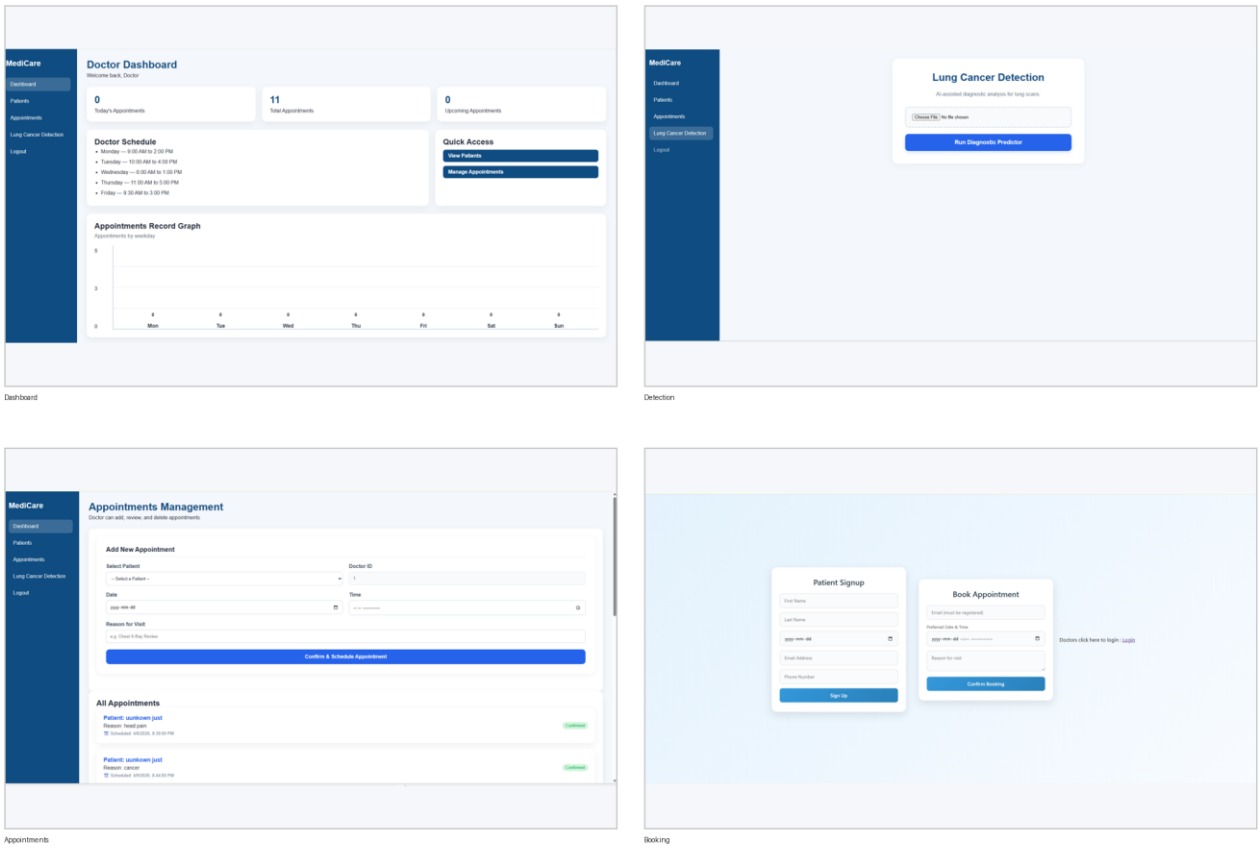

Figure 1. Application interface montage reconstructed from repository screenshots.

8. Conclusion

This report documents the project based on the uploaded repository itself, not on assumptions. On that basis, Medicare is a substantial capstone implementation with clear technical merit, meaningful AI integration, and a credible prototype workflow. Its main limitations are incomplete route synchronization and weak test evidence, both of which are normal and fixable in a late-stage student project. With stronger API consistency and repeatable validation, it could evolve from a solid demonstration into a more dependable engineering deliverable.